فصل ۹: درخت تصمیم و Ensembles

مقدمه

در این فصل، ما روی درختان تصمیم و الگوریتم‌های گروه (Ensembles) تمرکز می‌کنیم. تفسیر و تجسم الگوریتم‌های تصمیم‌گیری به آسانی امکان‌پذیر است زیرا آن‌ها رئوس مطالب فرایند تصمیم‌گیری است که با آن آشنا هستیم. گروه‌ها (ensembles) را می‌توان تا حدی تفسیر و تجسم کرد، اما قسمت‌های زیادی دارند (تخمین زننده پایه)، بنابراین ما همیشه نمی‌توانیم آن‌ها را به راحتی درک کنیم.

هدف از یادگیری گروه (ensemble) این است که چندین تخمین‌زننده می‌توانند بهتر از یک ارزیابی واحد کار کنند. برای این کار دو نوع روش وجود دارد: روش‌های میانگین‌گیری و روش‌های تقویتی. روش‌های میانگین‌گیری (random forest، bagging، extra trees) با میانگین‌گیری از پیش‌بینی‌های چندین تخمین زننده، واریانس را کاهش می‌دهند. روش‌های تقویتی (افزایش گرادیان و AdaBoost) با استفاده از برآوردگرهای پایه پی در پی، با هدف کاهش بایاس کل گروه، بایاس را کاهش می‌دهد.

ویژگی مشترک بسیاری از Ensemblesها استفاده از "تصادفی‌سازی" برای ساخت پیش‌بینی‌ها است. به عنوان مثال random forest از داده‌های تصادفی (همانطور که از نام آن پیداست) استفاده می‌کند، و از جستجو در میان پارامترهای زیاد، که از تصادفی بودن بهره می‌برند، استفاده خواهد کرد. از ایده‌های تصادفی بودن در این فصل برای ایجاد الگوریتم‌هایی با امتیاز بهتر استفاده می‌شود.

نکته:
از آنجا که محاسبات پر هزینه هستند، در صورت امکان، از یک کامپیوتر چند هسته‌ای استفاده کنید. در کل، یک پارامتر به نام n_jobs وجود دارد که اگر مقدار ۱- داشته باشد از تمام هسته‌های رایانه شما استفاده می‌کند.

کلاسه بندی‌های ابتدایی با درختان تصمیم

در اینجا، ما کلاسه‌بندی اساسی را با درختان تصمیم انجام می‌دهیم. درختان تصمیم برای کلاسه‌بندی توالی تصمیماتی هستند که طبقه‌بندی یا یک نتیجه طبقه‌بندی شده را تعیین می‌کنند.

با بارگذاری مجدد مجموعه iris و تقسیم داده‌ها به مجموعه‌های آموزشی و آزمایشی، شروع می‌کنیم:

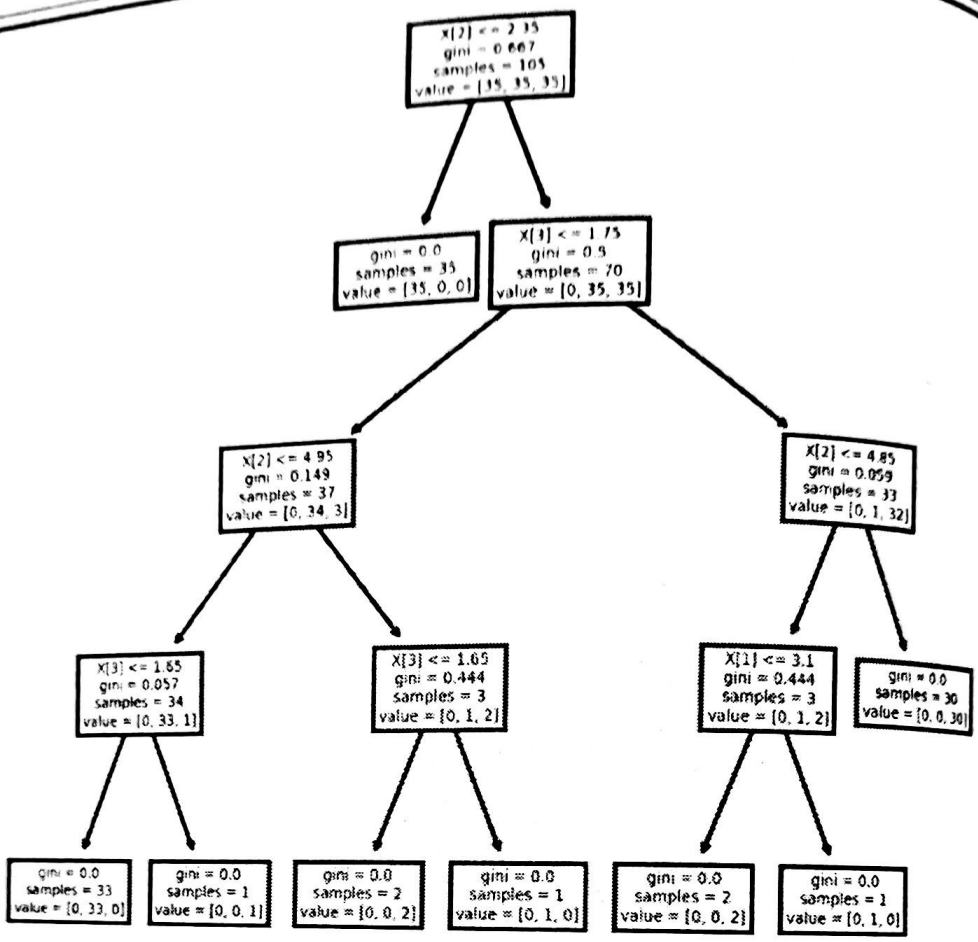
```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data
y = iris.target
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,stratify=y)
from sklearn.tree import DecisionTreeClassifier
dtc = DecisionTreeClassifier() #Instantiate tree class
dtc.fit(X_train, y_train)
```

حال دقت آن را مشاهده می‌کنیم

```
from sklearn.metrics import accuracy_score
y_pred = dtc.predict(X_test)
accuracy_score(y_test, y_pred)
output: 0.9777777777777777
```

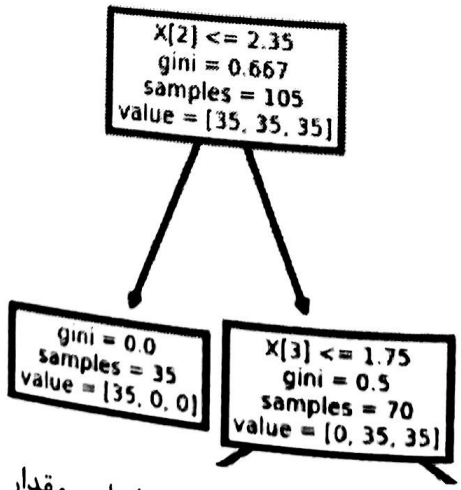
نمایش درخت به صورت زیر است

```
from sklearn import tree
import matplotlib.pyplot as plt
graph=tree.plot_tree(dtc);
```



شکل (۹-۱): نمایش درخت داده‌ها

این درخت تصمیمی است که با آموزش تولید شده است. از بالای درخت شروع به دیدن آن کنید. شما ۱۰۵ نمونه در مجموعه آموزش دارید. مجموعه آموزش به سه مجموعه ۳۵ تایی تقسیم می‌شود: مقدار = [۳۵، ۳۵، ۳۵]. به صراحت، اینها ۳۵ گل Setosa، ۳۵ گل Versicolor و ۳۵ گل Virginica است.



شکل (۹-۲): تقسیم داده‌ها بر اساس مقدار gini

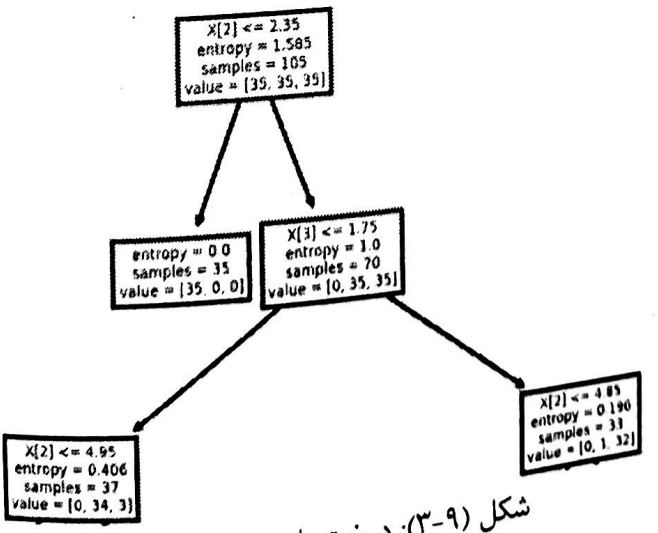
اولین تصمیم‌گیری این است که آیا طول گلبرگ گل کمتر یا مساوی ۲/۴۵ است یا خیر. جواب درست باشد، یا بله، گل در دسته اول طبقه‌بندی می‌شود، مقدار = [۰، ۰، ۳۵] این گل به عنوان یک گل Setosa طبقه‌بندی می‌شود. در چندین نمونه از طبقه‌بندی معمولی iris، این آسان‌ترین طبقه‌بندی بود. (فلش سمت راست بیانگر False و فلش سمت چپ بیانگر True است). در غیر این صورت، اگر طول گلبرگ از 2.45 بیشتر باشد، اولین تصمیم منجر به درخت تصمیم‌گیری کوچک‌تر می‌شود. درخت تصمیم کوچک‌تر فقط دارای گل‌هایی از دو نوع آخر، Versicolour و Virginica است و مقدار = [۰، ۳۵، ۳۵].

الگوریتم به تولید یک درخت کامل از چهار سطح، تا عمق ۴ ادامه می‌دهد (توجه داشته باشید که گره بالایی در شمارش سطوح در نظر گرفته نمی‌شود). به زبان رسمی، سه گره مشخص کننده یک تصمیم در تصویر، "تقسیم" نامیده می‌شوند.

ممکن است تعجب کنید که مرجع gini در نمودار درخت تصمیم چیست. جینی به عملکرد gini اشاره می‌کند، که کیفیت تقسیم را اندازه‌گیری می‌کند. هنگام اجرای الگوریتم، چند انشعاب که عملکرد gini را بهینه می‌کنند در نظر گرفته می‌شوند. بخشی که بهترین معیار gini را ایجاد می‌کند انتخاب می‌شود.

گزینه دیگر اندازه‌گیری آنتروپی برای تعیین نحوه تقسیم درخت است. با استفاده از اعتبارسنجی متقابل می‌توانید هر دو گزینه را امتحان کنید و تعیین کنید که کدام یک بهتر است. به مثال زیر توجه کنید:

```
from sklearn.tree import DecisionTreeClassifier
dtc = DecisionTreeClassifier(criterion='entropy')
dtc.fit(X_train, y_train)
fig, axes = plt.subplots(nrows = 1,ncols = 1,figsize = (4,4), dpi=300)
tree.plot_tree(dtc);
fig.savefig('entropy.png')
```



شکل (۹-۳): درخت با توجه به entropy

تنظیم درخت تصمیم

ما با تمرکز بر دو ویژگی اول (طول و عرض کاسبرگ)، بهینه‌سازی درخت تصمیم، به اکتشاف مجموعه داده iris ادامه خواهیم داد.

با تمرکز بر روی دو ویژگی اول، مجموعه iris را بارگیری کنید. علاوه بر این، داده‌ها را به مجموعه‌های آموزش و آزمایش تقسیم کنید:

```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data[:,:2]
y = iris.target
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.3, stratify=y)
import pandas as pd
pd.DataFrame(X,columns=iris.feature_names[:2])
```

	sepal length (cm)	sepal width (cm)
0	5.1	3.5
1	4.9	3.0
2	4.7	3.2
3	4.6	3.1
4	5.0	3.6
...	...	...
145	6.7	3.0
146	6.3	2.5
147	6.5	3.0
148	6.2	3.4
149	5.9	3.0

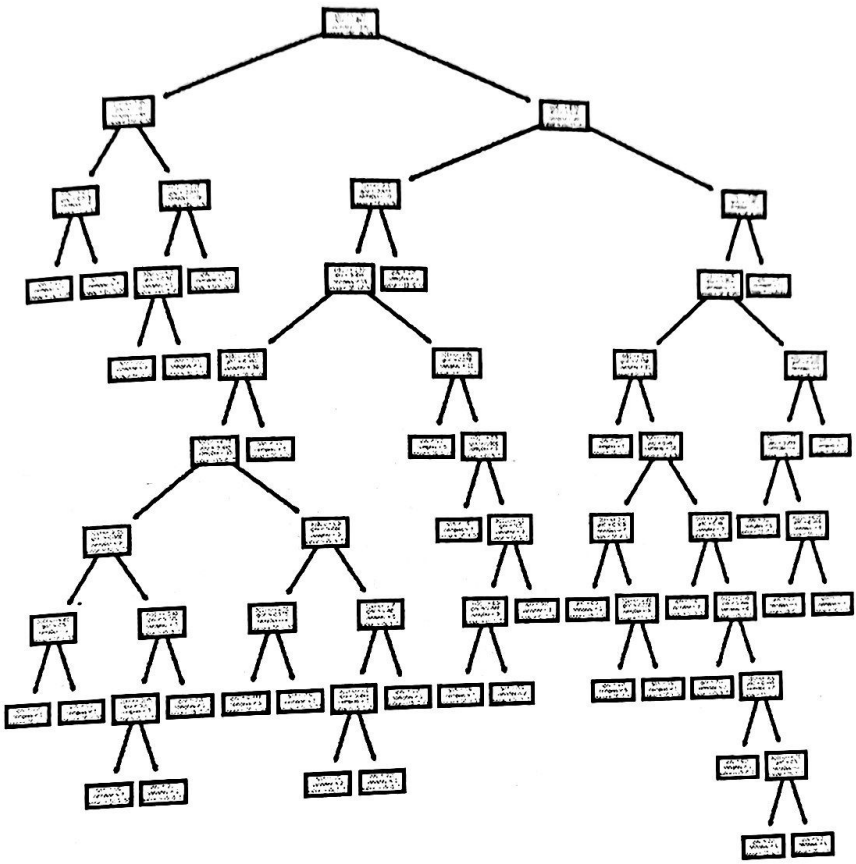
150 rows × 2 columns

شکل (۹-۴): داده‌های دو ویژگی اول

قبل از بهینه‌سازی درخت تصمیم‌گیری، بیایید یک درخت تصمیم را با پارامترهای پیش فرض امتحان کنیم.

```
from sklearn.tree import DecisionTreeClassifier
dtc = DecisionTreeClassifier() #Instantiate tree with default parameters
dtc.fit(X_train, y_train)
```

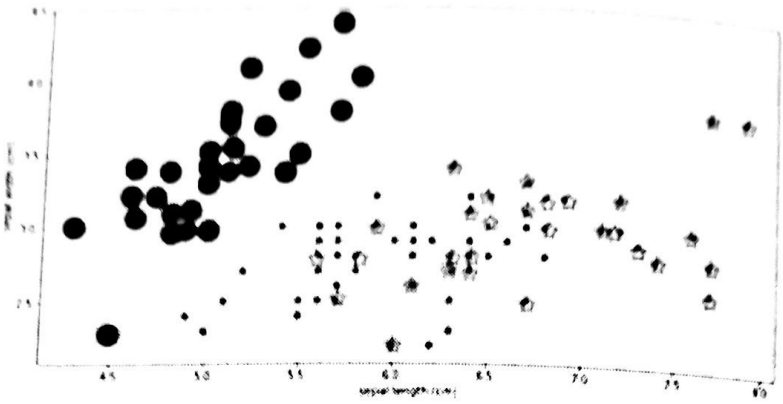
```
from sklearn.metrics import accuracy_score
y_pred = dtc.predict(X_test)
accuracy_score(y_test, y_pred)
output: 0.6
```



شکل (۹-۵): درخت با بیش برازش

این بزرگی و وسعت به خاطر بیش برازش است به عبارت دیگر درخت تصمیم بسیار مفصل است. کل مجموعه iris شامل ۱۵۰ نمونه است و یک درخت بسیار پیچیده نامطلوب است. به یاد بیاورید که در فصل‌های قبلی ما از SVMهای خطی استفاده کردیم که فضا را به روشی ساده با چند خط تقسیم می‌کند.

```
import matplotlib.pyplot as plt
%matplotlib inline
plt.figure(figsize=((12,6)))
plt.xlabel(iris.feature_names[0])
plt.ylabel(iris.feature_names[1])
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train )
```



شکل (۹-۶): طول بر حسب عرض گلبرگ‌ها

برای بهینه‌سازی عملکرد درخت تصمیم، از GridSearchCV استفاده می‌شود.

```
from sklearn.tree import DecisionTreeClassifier
dtc = DecisionTreeClassifier()
from sklearn.model_selection import GridSearchCV, cross_val_score
param_grid = {'criterion':['gini','entropy'], 'max_depth' :[3,5,7,20]}
gs_inst = GridSearchCV(dtc,param_grid=param_grid,cv=5)
gs_inst.fit(X_train, y_train)
```

توجه داشته باشید که چگونه در پارامتر شبکه، param_grid، معیار امتیازدهی تقسیم را بین جینی و انتروپی تغییر می‌دهیم و حداکثر عمق یک درخت را تغییر می‌دهیم.

حال دقت آن را نمایش می‌دهیم

```
from sklearn.metrics import accuracy_score
y_pred_gs = gs_inst.predict(X_test)
accuracy_score(y_test, y_pred_gs)
output: 0.7777777777777778
```

برای مشاهده جزییات از cv_results_ استفاده نمایید

```
print(list(zip(gs_inst.cv_results_['mean_test_score'],gs_inst.cv_results_['params'])))
output: [(0.7619047619047619, {'criterion': 'gini', 'max_depth': 3}),
(0.7523809523809524, {'criterion': 'gini', 'max_depth': 5}),
(0.7142857142857142, {'criterion': 'gini', 'max_depth': 7}),
(0.7333333333333334, {'criterion': 'gini', 'max_depth': 20}),
(0.7523809523809523, {'criterion': 'entropy', 'max_depth': 3}),
(0.7333333333333333, {'criterion': 'entropy', 'max_depth': 5}),
(0.6952380952380952, {'criterion': 'entropy', 'max_depth': 7}),
(0.7238095238095238, {'criterion': 'entropy', 'max_depth': 20})]
```

از طریق لیست امتیازات می‌توان مشاهده کرد که درختان عمیق‌تر عملکرد کمتری نسبت به درختان کم عمق دارند. به طور جزئی، داده‌های مجموعه آموزشی به پنج قسمت تقسیم

می‌شوند. آموزش در چهار قسمت اتفاق می‌افتد در حالی که آزمایش در یکی از پنج قسمت اتفاق می‌افتد. درختان بسیار عمیق دارای بیش برازش هستند: آن‌ها در مجموعه‌های آموزشی عملکرد خوبی دارند، اما در پنج مجموعه تست اعتبارسنجی عملکرد بدی دارند.

بهترین درخت را با best_estimator_ انتخاب کنید

```
gs_inst.best_estimator_
output: DecisionTreeClassifier(max_depth=3)
```

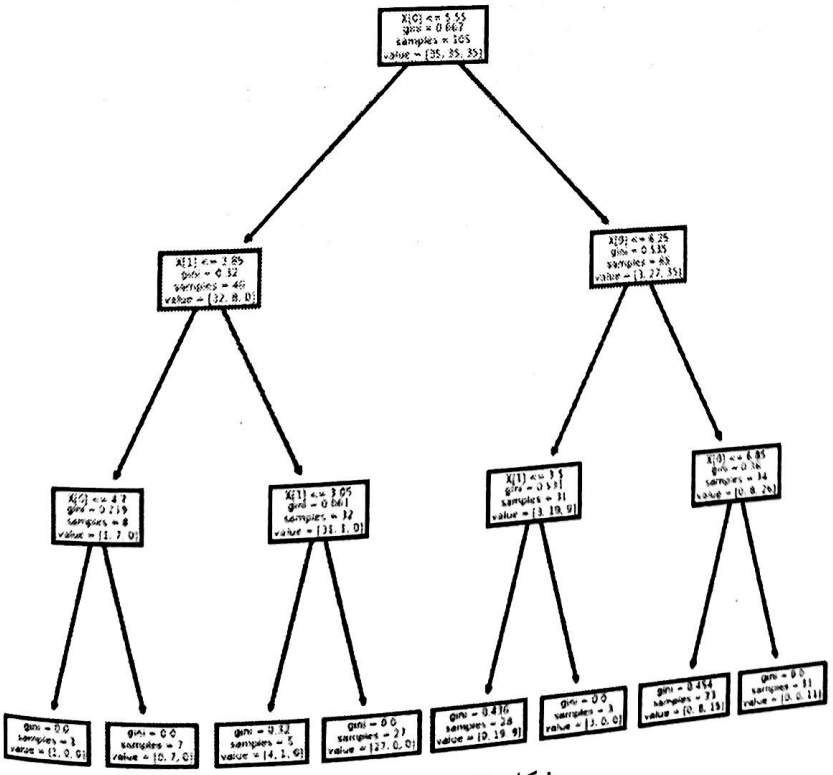
بهترین انتخاب را نمایش می‌دهیم

```
fig, axes = plt.subplots(nrows = 1,ncols = 1,figsize = (4,4), dpi=300)
tree.plot_tree(gs_inst.best_estimator_);
fig.savefig('gridsearch.png')
```

نمودار آن به صورت شکل ۹-۷ خواهد بود.

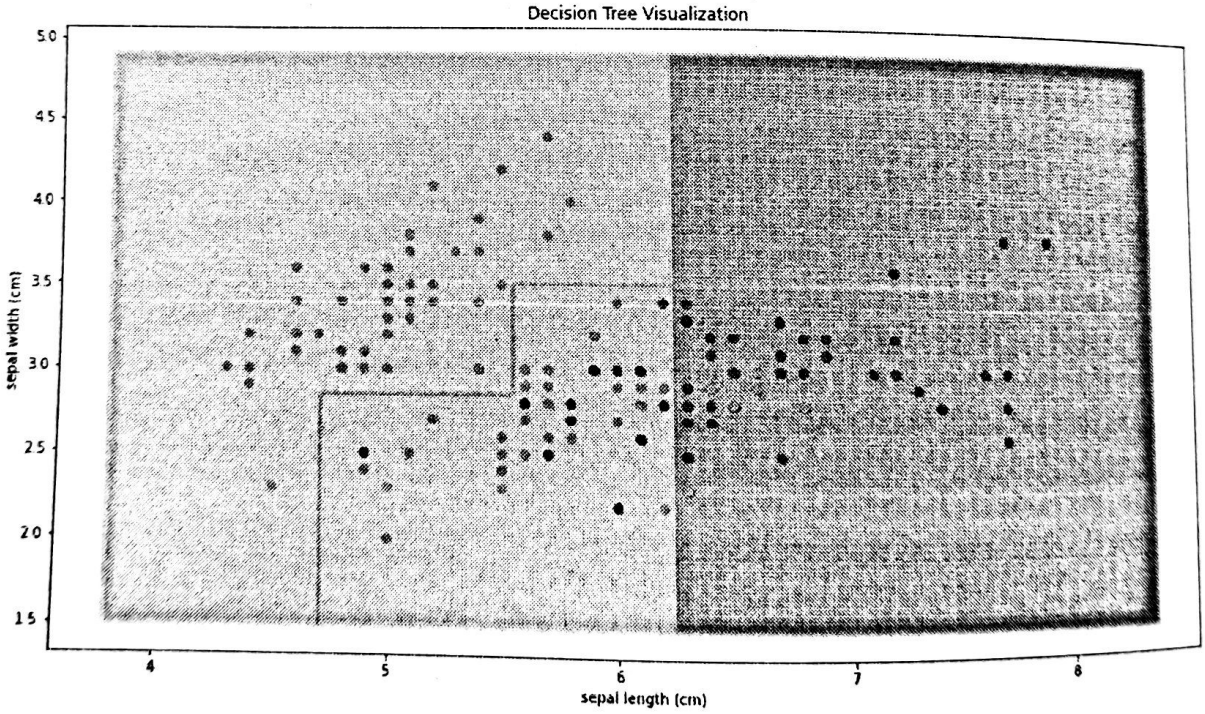
حال داده‌ها را به شکل دیگری نمایش می‌دهیم مانند شکل (۹-۸)

```
grid_interval = 0.02
x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
xmin, xmax = np.percentile(X[:, 0], [0, 100])
ymin, ymax = np.percentile(X[:, 1], [0, 100])
xmin_plot, xmax_plot = xmin - .5, xmax + .5
ymin_plot, ymax_plot = ymin - .5, ymax + .5
```



شکل (۹-۷): درخت داده‌ها

```
xx, yy = np.meshgrid(np.arange(xmin_plot,
xmax_plot,grid_interval),np.arange(ymin_plot, ymax_plot, grid_interval))
test_preds =gs_inst.best_estimator_.predict(list(zip(xx.ravel(),yy.ravel())))
import matplotlib.pyplot as plt
%matplotlib inline
X_0 = X[y == 0]
X_1 = X[y == 1]
X_2 = X[y == 2]
plt.figure(figsize=(15,8)) #change figure-size for easier viewing
plt.scatter(X_0[:,0],X_0[:,1], color = 'red')
plt.scatter(X_1[:,0],X_1[:,1], color = 'blue')
plt.scatter(X_2[:,0],X_2[:,1], color = 'green')
colors = np.array(['r', 'b','g'])
plt.scatter(xx.ravel(), yy.ravel(), color=colors[test_preds],alpha=0.15)
plt.scatter(X[:, 0], X[:, 1], color=colors[y])
plt.title("Decision Tree Visualization")
plt.xlabel(iris.feature_names[0])
plt.ylabel(iris.feature_names[1])
```

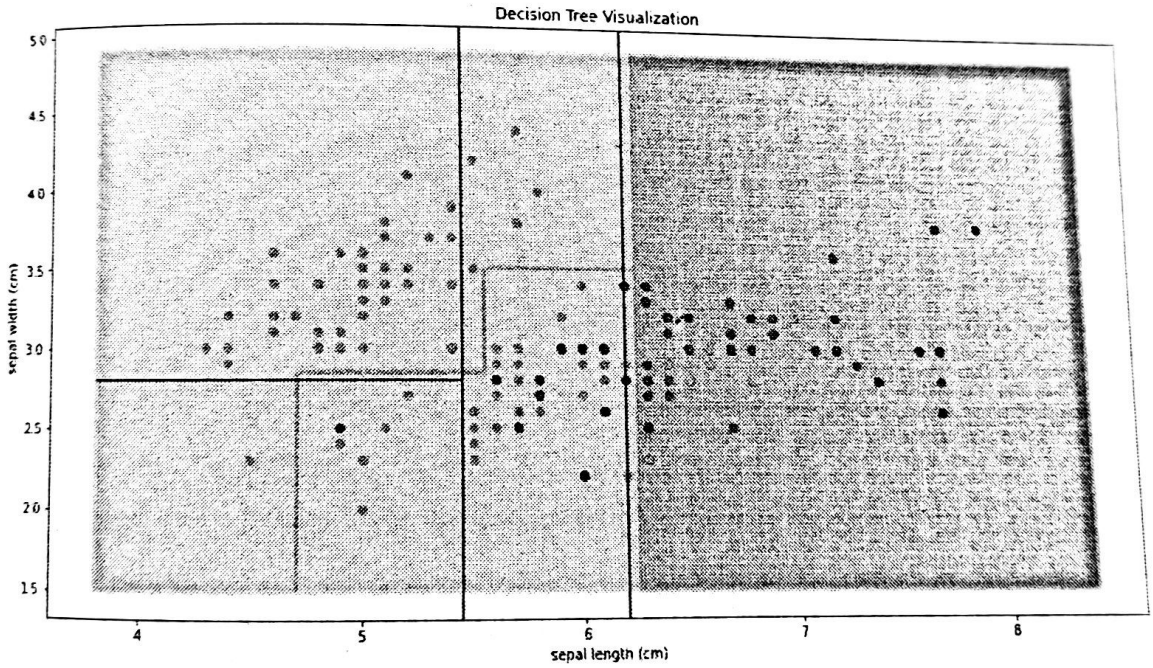


شکل (۹-۸): مرز تصمیم بین داده‌ها

در این تصویر، می‌بینید که درختان تصمیم برای طبقه‌بندی نوع گل iris سعی در ساخت مستطیل دارند. هر بخش یک خط عمود بر یکی از ویژگی‌ها ایجاد می‌کند. در نمودار زیر یک خط عمودی وجود دارد که اولین تصمیم به این صورت گرفته می‌شود که

اگر طول کاسبرگ از عدد ۵/۴۵ بیشتر باشد، سمت راست خط را نشان می‌دهد و اگر کمتر باشد، سمت چپ خط را شامل می‌شود.

```
plt.figure(figsize=(15,8)) #change figure-size for easier viewing
plt.scatter(X_0[:,0],X_0[:,1], color = 'red')
plt.scatter(X_1[:,0],X_1[:,1], color = 'blue')
plt.scatter(X_2[:,0],X_2[:,1], color = 'green')
colors = np.array(['r', 'b','g'])
plt.scatter(xx.ravel(), yy.ravel(), color=colors[test_preds],alpha=0.15)
plt.scatter(X[:, 0], X[:, 1], color=colors[y])
plt.title("Decision Tree Visualization")
plt.xlabel(iris.feature_names[0])
plt.ylabel(iris.feature_names[1])
plt.axvline(x = 5.45, color='black')
plt.axvline(x = 6.2, color='black')
plt.plot((xmin_plot, 5.45), (2.8, 2.8), color='black')
```



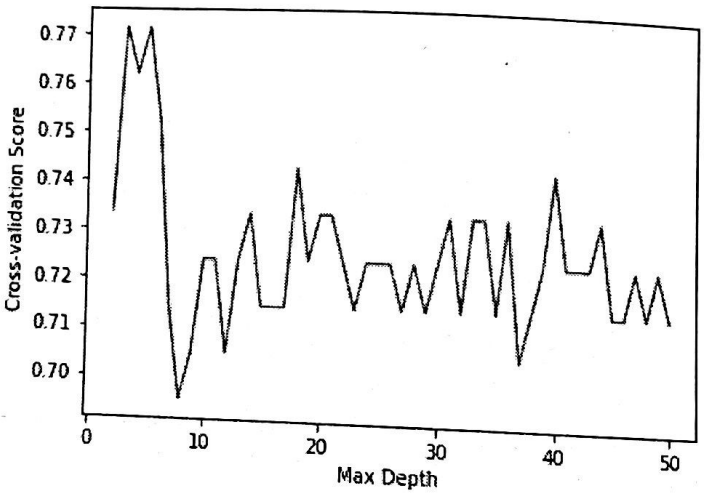
شکل (۹-۹): مرزهای تصمیم ابتدایی

خط افقی، sepal_width = 2.8، کوتاهتر است و به x = 5.45 ختم می‌شود.

در پایان، چندین منطقه مستطیل شکل ایجاد می‌شود. نمودار زیر همان نوع نمودار را نشان می‌دهد که برای درخت تصمیم بسیار بزرگ که دارای بیش برازش است اعمال شده است. درخت تصمیم یک مستطیل را در اطراف بسیاری از نمونه‌های خاص مجموعه داده iris قرار می‌دهد، که نشان دهنده تعمیم ضعیف است.

ما همچنین می‌توانیم چگونگی تأثیر حداکثر عمق روی امتیاز اعتبار متقابل را ترسیم کنیم جستجوی شبکه را با حداکثر عمق از ۲ تا ۵۱ نمایش می‌دهیم

```
from sklearn.tree import DecisionTreeClassifier
dtc = DecisionTreeClassifier()
from sklearn.model_selection import GridSearchCV, cross_val_score
max_depths = range(2,51)
param_grid = {'max_depth' : max_depths}
gs_inst = GridSearchCV(dtc, param_grid=param_grid,cv=5)
gs_inst.fit(X_train, y_train)
plt.plot(max_depths,gs_inst.cv_results_['mean_test_score'])
plt.xlabel('Max Depth')
plt.ylabel("Cross-validation Score")
```



شکل (۹-۱۰): اعتبارسنجی متقابل بر اساس عمق درخت

نمودار نشان می‌دهد که عمق زیاد باعث کاهش امتیاز اعتبار متقابل می‌شود.

استفاده از درختان تصمیم برای رگرسیون

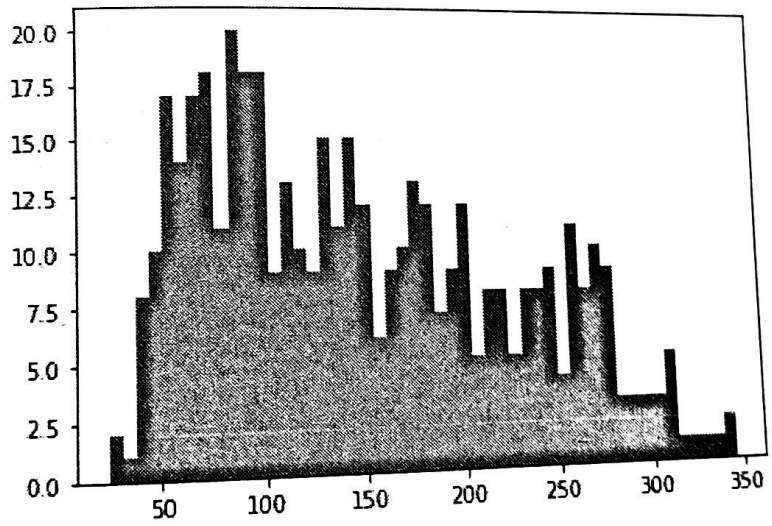
درختان تصمیم برای رگرسیون بسیار شبیه درختان تصمیم برای کلاسه‌بندی هستند.

روش تدوین مدل رگرسیون از چهار قسمت تشکیل شده است:

- مجموعه داده را بارگیری کنید
- مجموعه را به زیر مجموعه‌های آموزش / آزمایش تقسیم کنید

- یک درخت تصمیم برای آموزش مجدد آن تهیه کنید
- امتیاز مدل را در زیرمجموعه آزمون قرار دهید

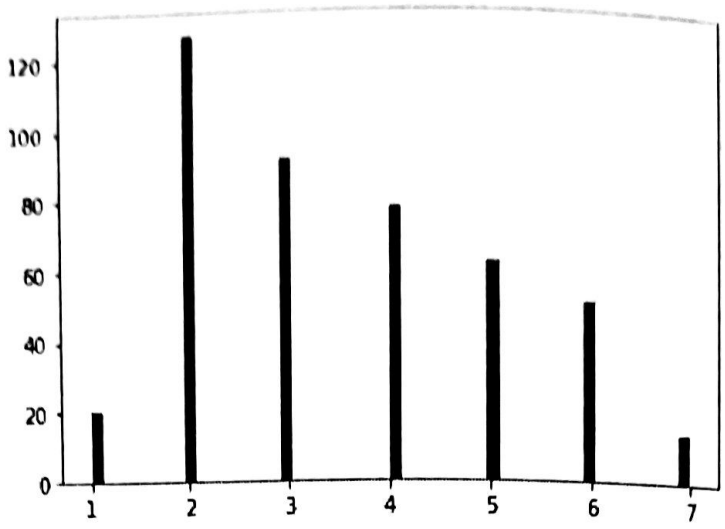
```
%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target
X_feature_names = ['age', 'gender', 'body mass index', 'average blood
pressure','bl_0','bl_1','bl_2','bl_3','bl_4','bl_5']
plt.hist(y , bins=50)
```



شکل (۹-۱۱): مقادیر هدف دیابت

این یک نمونه رگرسیون است و ما نمی‌توانیم هنگام تقسیم مجموعه داده از stratify = y استفاده کنیم. در مقابل، ما پیگیری خواهیم کرد که آیا متغیر هدف کمتر از ۵۰ است، یا بین ۵۰ تا ۱۰۰ است، و غیره

```
bins = 50*np.arange(8)
bins
array([ 0, 50, 100, 150, 200, 250, 300, 350])
binned_y = np.digitize(y, bins)
plt.hist(binned_y , bins=50)
```



شکل (۹-۱۲): دادهای هدف در سطل‌های متفاوت

آرایه binned_y پیگیری می‌کند که هر عنصر متعلق به کدام سطل است. اکنون، مجموعه را به مجموعه‌های آموزشی و آزمایشی تقسیم کنید و آرایه bins (سطل ها) را طبقه‌بندی کنید

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2,stratify=binned_y)
```

برای ایجاد رگرسیون درخت تصمیم، یک نمونه(شی) از آن ایجاد کرده و آموزش دهید

```
from sklearn.tree import DecisionTreeRegressor
dtr = DecisionTreeRegressor()
dtr.fit(X_train, y_train)
```

حال داده‌های تست را پیش‌بینی می‌کنیم

```
y_pred = dtr.predict(X_test)
```

برای مقایسه y_test و y_pred از معیار خطا استفاده می‌کنیم؛ در اینجا، از mean_absolute_error استفاده کنید، که میانگین مقدار مطلق اختلافات بین عناصر y_test و y_pred است

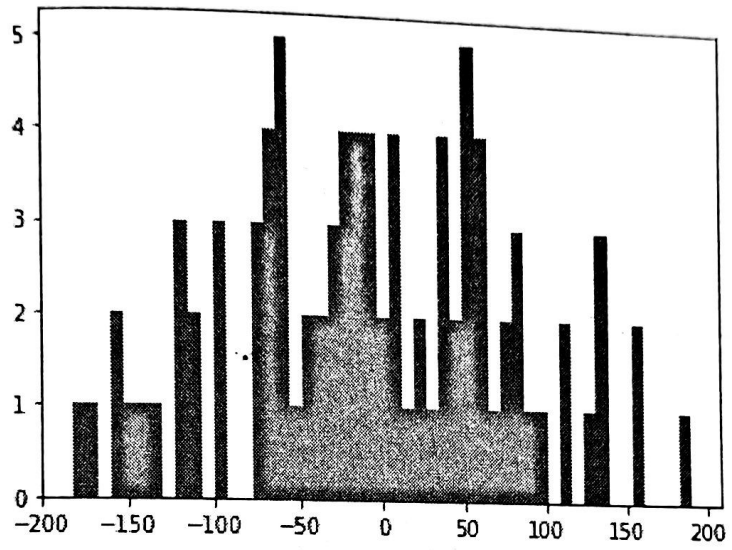
```
from sklearn.metrics import mean_absolute_error
mean_absolute_error(y_test, y_pred)
67.64044943820225
```

به عنوان یک گزینه، میانگین خطای درصد مطلق را که میانگین مقدار مطلق اختلافات تقسیم بر اندازه عناصر تست است، اندازه‌گیری کنید. این عمل خطا را نسبت به اندازه عنصر تست اندازه‌گیری می‌کند:

```
(np.abs(y_test - y_pred)/(y_test)).mean()
0.6246248998288833
```

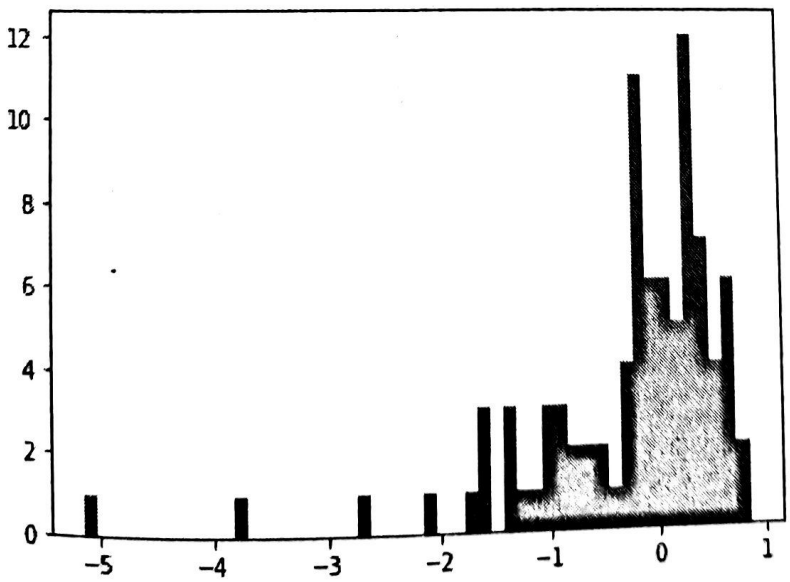
بنابراین، ما یک پایه عملکرد با توجه به مجموعه داده‌های دیابت ایجاد کرده‌ایم. هر تغییر در مدل احتمالاً روی اندازه‌گیری خطا تأثیر می‌گذارد.

```
plt.hist((y_test - y_pred) , bins=50)
```



شکل (۹-۱۳): تفاوت پیش‌بینی و داده‌های تست

```
plt.hist((y_test - y_pred) /(y_test), bins=50)
```



شکل (۹-۱۴): تفاوت داده تست و پیش‌بینی

کاهش بیش برازش با اعتبارسنجی متقابل

در اینجا، برای بهبود عملکرد، از اعتبارسنجی متقابل استفاده خواهیم کرد.

```
%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target
X_feature_names = ['age', 'gender', 'body mass index', 'average blood
pressure','bl_0','bl_1','bl_2','bl_3','bl_4','bl_5']
bins = 50*np.arange(8)
binned_y = np.digitize(y, bins)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,
y,test_size=0.2,stratify=binned_y)
```

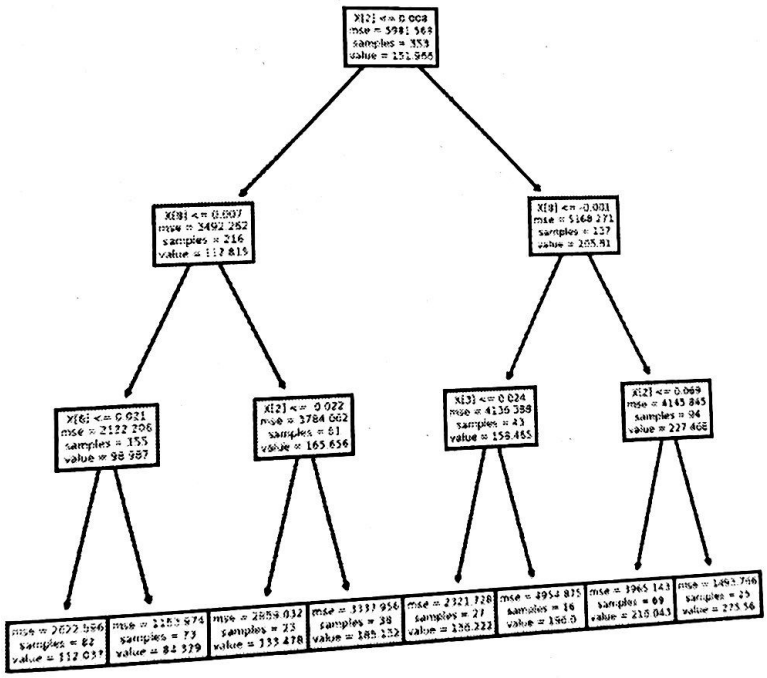
برای کاهش بیش برازش از جستجوی شبکه استفاده می‌کنیم.

```
from sklearn.tree import DecisionTreeRegressor
dtr = DecisionTreeRegressor()
from sklearn.model_selection import GridSearchCV
gs_inst = GridSearchCV(dtr, param_grid = {'max_depth':[3,5,7,9,20]},cv=10)
gs_inst.fit(X_train, y_train)
gs_inst.best_estimator_
DecisionTreeRegressor(criterion='mse', max_depth=3,
max_features=None,
max_leaf_nodes=None, min_impurity_split=1e-07,
min_samples_leaf=1, min_samples_split=2,min_weight_fraction_leaf=0.0,
presort=False, random_state=None,splitter='best')
y_pred = gs_inst.predict(X_test)
from sklearn.metrics import mean_absolute_error
mean_absolute_error(y_test, y_pred)
55.116768991403866
(np.abs(y_test - y_pred)/(y_test)).mean()
0.46028657266092077
```

نمایش آن به صورت شکل-۹-۱۵ خواهد بود.(این درخت دارای معیارهای دقت بهتری است و بیش برازش کمتری دارد.)

رگرسیون با جنگل تصادفی

جنگل‌های تصادفی یک الگوریتم Ensemble است. این الگوریتم‌ها برای بهبود پیش‌بینی‌ها از ترکیب چندین الگوریتم با هم استفاده می‌کنند. Scikit-learn چندین الگوریتم در این زمینه دارد که بیشتر آن‌ها از درختان برای پیش‌بینی استفاده می‌کنند. بیایید با گسترش رگرسیون درخت تصمیم با چندین درخت تصمیم که با هم در یک جنگل تصادفی کار می‌کنند شروع کنیم. جنگل تصادفی ترکیبی از چندین درخت تصمیم است، که در آن هر درخت یک رأی واحد نسبت به پیش‌بینی نهایی ارائه می‌دهد. جنگل تصادفی نهایی با میانگین‌گذاری نتایج تمام درختانی که از آن‌ها تشکیل شده است، یک خروجی نهایی را محاسبه می‌کند.



شکل (۹-۱۵): درخت حاصل پس از اعتبارسنجی متقابل

برای مثال، مجموعه داده‌های رگرسیون دیابت را همانطور که برای درختان تصمیم‌گیری انجام دادیم استفاده می‌کنیم. همه داده‌ها را به مجموعه‌های آموزشی و آزمایشی تقسیم کنید

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
```

```
diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target
X_feature_names = ['age', 'gender', 'body mass index', 'average blood
pressure','bl_0','bl_1','bl_2','bl_3','bl_4','bl_5']
#bin target variable for better sampling
bins = 50*np.arange(8)
binned_y = np.digitize(y, bins)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2,stratify=binned_y)
```

جنگل را فراخوانی و پیش‌بینی را انجام می‌دهیم

```
from sklearn.ensemble import RandomForestRegressor
rft = RandomForestRegressor()
rft.fit(X_train, y_train)
y_pred = rft.predict(X_test)
from sklearn.metrics import mean_absolute_error
print(mean_absolute_error(y_test, y_pred))
print((np.abs(y_test - y_pred)/(y_test)).mean())
```

```
output:
45.384494382022474
0.42357504363326437
```

خطاها در مقایسه با یک درخت تصمیم کمی کاهش یافته است.

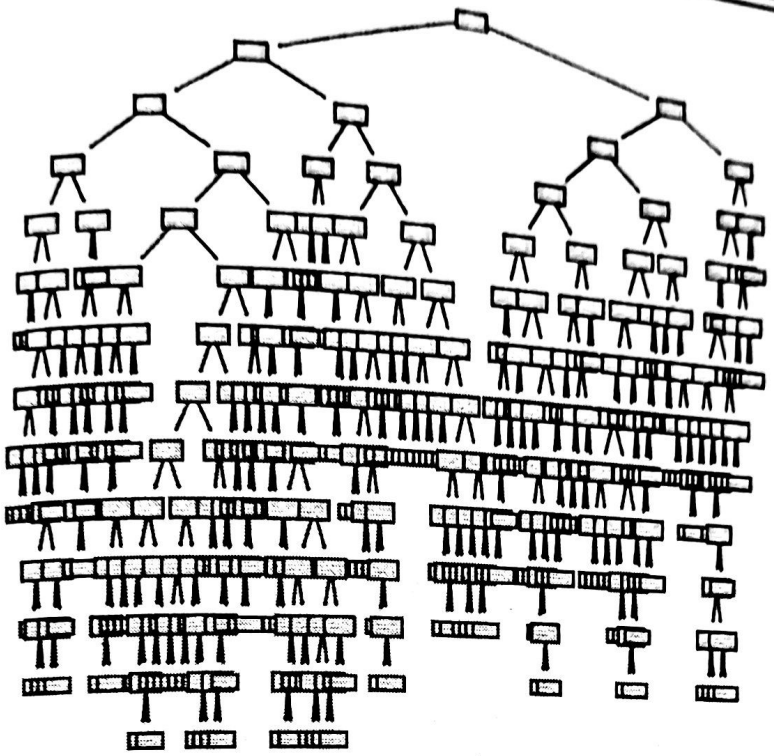
برای مشاهده پارامترها از _estimators استفاده نمایید

```
rft.estimators_
```

```
output:
[DecisionTreeRegressor(max_features='auto', random_state=1123756894),
 DecisionTreeRegressor(max_features='auto', random_state=2085801587),
 DecisionTreeRegressor(max_features='auto', random_state=847732974),
 ......
```

برای مشاهده اولین درخت از لیست در نمودار، از اولین عنصر در لیست rft.estimators_[0] استفاده کنید:

```
fig, axes = plt.subplots(nrows = 1,ncols = 1,figsize = (4,4), dpi=300)
tree.plot_tree(rft.estimators_[0]);
fig.savefig('randomforest1.png')
```



شکل (۹-۱۶): اولین درخت

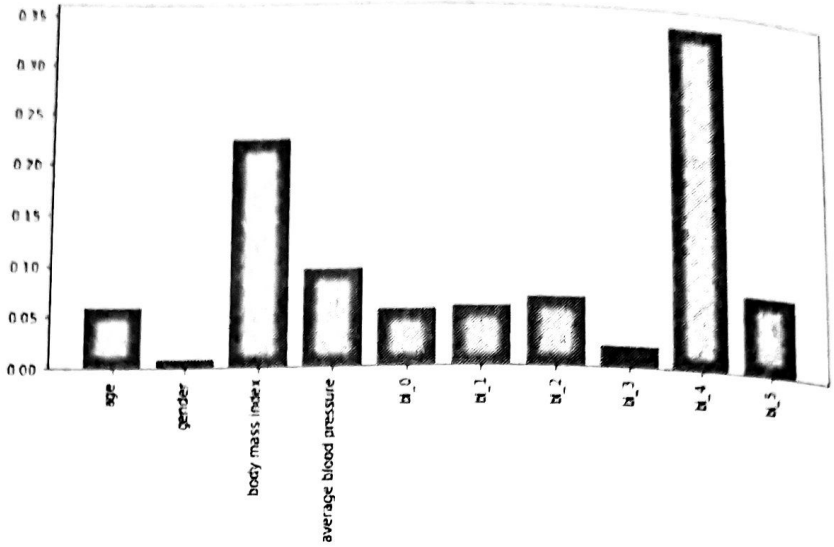
برای مشاهده درخت دوم، از best_rft.estimators_ [1] استفاده کنید.

یکی دیگر از ویژگی‌های جنگل تصادفی تعیین اهمیت ویژگی از طریق feature_importances_attribute است

```
rft.feature_importances_
output: array([0.05814878, 0.00830414, 0.22312792, 0.09353004, 0.05443527,
  0.05657576, 0.06689391, 0.01960509, 0.34307547, 0.07630363])
```

نمایش ویژگی‌های مهم به صورت زیر است

```
fig, ax = plt.subplots(figsize=(10,5))
bar_rects = ax.bar(np.arange(10),
rft.feature_importances_,color='r',align='center')
ax.xaxis.set_ticks(np.arange(10))
ax.set_xticklabels(X_feature_names, rotation='vertical')
```



شکل (۹-۱۷): امتیاز هر ویژگی

مهم‌ترین ویژگی‌ها bl_4، سپس body mass index (BMI) و به دنبال آن bl_ فشار خون متوسط است.

رگرسیون bagging با نزدیک‌ترین همسایه

bagging یک نوع Ensemble است که جالب است که لزوماً شامل درختان نمی‌شود. چندین نمونه از یک تخمین زننده پایه را ایجاد می‌کند که بر روی زیر مجموعه‌های تصادفی از اولین داده‌های آموزشی کار می‌کند .در این بخش از KNN به عنوان تخمین زننده پایه استفاده می‌کنیم. از نظر عملی، bagging برای کاهش واریانس یک تخمین زننده پیچیده- به عنوان مثال یک درخت تصمیم با سطوح زیاد- عالی است. از طرف دیگر، boosting باعث کاهش بایاس مدل‌های ضعیف، مانند درختان تصمیم‌گیری در سطح بسیار کم، یا مدل‌های خطی می‌شود.

برای امتحان کردن baggin، ما بهترین پارامترها را پیدا می‌کنیم، جستجوی ابر پارامتر، با استفاده از جستجوی تصادفی شبکه‌ای امکان‌پذیر است. برای این هدف مراحل زیر را طی خواهیم کرد:

۱. مشخص می‌کنیم که کدام پارامترها را در الگوریتم بهینه کنیم (این پارامترهایی است که محققان بهترین آن‌ها را برای بهینه‌سازی در مقالات می‌دانند).

۲. توزیع پارامتر را در جایی که مهم‌ترین پارامترها هستند انجام دهید.
۳. جستجوی شبکه‌ای تصادفی را انجام دهید. اگر از یک گروه استفاده می‌کنید، ابتدا تعداد تخمین زننده‌ها را کم نگه دارید.
۴. از بهترین پارامترهای مرحله قبل با بسیاری از تخمین زننده‌ها استفاده کنید.

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_diabetes
diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target
X_feature_names = ['age', 'gender', 'body mass index', 'average
bloodpressure','bl_0','bl_1','bl_2','bl_3','bl_4','bl_5']
#bin target variable for better sampling
bins = 50*np.arange(8)
binned_y = np.digitize(y, bins)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,
y,test_size=0.2,stratify=binned_y)
```

ماژول‌های مورد نیاز را انتخاب می‌کنیم. سپس، یک توزیع پارامتر برای جستجوی شبکه تنظیم کنید. برای bagging، برخی از پارامترها برای تغییر شامل max_samples، max_features، oob_score و تعداد تخمین زننده، n_estimators هستند. تعداد تخمین زننده‌ها را روی تعداد کم ۱۰۰ قرار داده شده است تا سایر پارامترها را بهینه کند.

```
from sklearn.ensemble import BaggingRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import RandomizedSearchCV
param_dist = {
'max_samples': [0.5,1.0],
'max_features' : [0.5,1.0],
'oob_score' : [True, False],
'base_estimator__n_neighbors': [3,5],
'n_estimators': [100]}
```

یک شی KNeighborsRegressor ایجاد و سپس آن را به BaggingRegressor ارسال می‌کنیم

```
single_estimator = KNeighborsRegressor()
ensemble_estimator = BaggingRegressor(base_estimator =single_estimator)
```

جستجوی تصادفی را با ۵ بار تکرار انجام می‌دهیم

```
pre_gs_inst_bag =
RandomizedSearchCV(ensemble_estimator,param_distributions = param_dist,
cv=3,n_iter = 5,n_jobs=1)
pre_gs_inst_bag.fit(X_train, y_train)
```

حال می‌توان بهترین پارامتر را مشاهده نمود

```
pre_gs_inst_bag.best_params_
output: {'oob_score': False,
 'n_estimators': 100,
 'max_samples': 1.0,
 'max_features': 0.5,
 'base_estimator__n_neighbors': 3}
```

حال تعداد تخمین زننده‌ها را به ۱۰۰۰ افزایش می‌دهیم

```
rs_bag = BaggingRegressor(**{'max_features': 1.0,
'max_samples': 0.5,
'n_estimators': 1000,
'oob_score': True,
'base_estimator': KNeighborsRegressor(n_neighbors=5)})
rs_bag.fit(X_train, y_train)
```

در آخر، عملکرد را روی یک مجموعه آزمایشی اندازه‌گیری کنید. الگوریتم مانند سایر موارد عملکرد خوبی ندارد، اما بعداً می‌توانیم از آن به عنوان بخشی از جمع کننده پشته استفاده کنیم

```
y_pred = rs_bag.predict(X_test)
from sklearn.metrics import r2_score, mean_absolute_error
print ("R-squared",r2_score(y_test, y_pred))
print ("MAE : ",mean_absolute_error(y_test, y_pred))
print ("MAPE : ",(np.abs(y_test - y_pred)/y_test).mean())
output:
R-squared 0.4402754526473577
MAE : 47.014029213483134
MAPE : 0.4241274289096285
```

اگر با دقت نگاه کنید، رگرسیون bagging کمی بهتر از جنگل تصادفی در بخش قبلی عمل می‌کند زیرا هر دو میانگین خطای مطلق و میانگین خطای درصد مطلق بهتر هستند. همیشه به یاد داشته باشید که لازم نیست Ensemble را برای یادگیری به درختان محدود کنید (از سایر الگوریتم‌ها می‌توانید استفاده نمایید) در اینجا، شما یک الگوریتم دیگر را با الگوریتم KNN ایجاد می‌کنید.

تنظیم درختان تقویت‌کننده

ما مجموعه داده‌های مسکن کالیفرنیا را با درختان تقویت کننده بررسی خواهیم کرد.
رویکرد کلی ما همان روش قبلی خواهد بود
تمرکز بر پارامترهای مهم در الگوریتم افزایش گرادیان است:

- algorithm
- max_features
- max_depth
- min_samples_leaf
- learning_rate
- loss

- توزیع پارامتر را در جایی که مهمترین پارامترها متنوع هستند ایجاد کنید.
- جستجوی شبکه‌ای تصادفی را انجام دهید. اگر از یک گروه استفاده می‌کنید، در ابتدا تعداد تخمین زننده‌ها را کم نگه دارید.
- از بهترین پارامترهای مرحله قبل با بسیاری از تخمین زننده استفاده کنید.

```
from __future__ import division
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
cali_housing = fetch_california_housing()
X = cali_housing.data
y = cali_housing.target
bins = np.arange(6)
binned_y = np.digitize(y, bins)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,
y,test_size=0.2,stratify=binned_y)
```

الگوریتم تقویت گرادیان و جستجوی تصادفی شبکه را استفاده نمایید

```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import RandomizedSearchCV
param_dist = {'max_features' : ['log2',1.0],
'max_depth' : [3, 5, 7, 10],
'min_samples_leaf' : [2, 3, 5, 10],
'n_estimators': [50, 100],
'learning_rate' : [0.0001,0.001,0.01,0.05,0.1,0.3],
'loss' : ['ls','huber']}
```

```
pre_gs_inst
=RandomizedSearchCV(GradientBoostingRegressor(warm_start=True),
param_distributions = param_dist,cv=3,n_iter = 30, n_jobs=1)
pre_gs_inst.fit(X_train, y_train)
```

حال گزارشی از آن تهیه می‌کنیم

```
import numpy as np
import pandas as pd
def get_grid_df(fitted_gs_estimator):
  res_dict = fitted_gs_estimator.cv_results_
  results_df = pd.DataFrame()
  for key in res_dict.keys():
    results_df[key] = res_dict[key]
  return results_df
def group_report(results_df):
  param_cols = [x for x in results_df.columns if 'param' in x and x is not 'params']
  focus_cols = param_cols + ['mean_test_score']
  print ("Grid CV Report \n")
  output_df = pd.DataFrame(columns = ['param_type','param_set',
'mean_score','mean_std'])
  cc = 0
  for param in param_cols:
    for key,group in results_df.groupby(param):
      output_df.loc[cc] = (param, key,
      group['mean_test_score'].mean(), group['mean_test_score'].std())
      cc += 1
  return output_df
```

شکل (۹-۱۸) نمایش جدول تخمین‌ها می‌باشد.

با استفاده از این اطلاعات، جستجوی تصادفی دیگری را امتحان می‌کینم

```
param_dist = {'max_features' : ['sqrt',0.5,1.0],
'max_depth' : [2,3,4],
'min_samples_leaf' : [3, 4];
'n_estimators': [50, 100],'learning_rate' : [0.2,0.25, 0.3, 0.4],
'loss' : ['ls','huber']
}
pre_gs_inst
=RandomizedSearchCV(GradientBoostingRegressor(warm_start=True),
param_distributions = param_dist,cv=3,n_iter = 30, n_jobs=1)
pre_gs_inst.fit(X_train, y_train)
results_df = get_grid_df(pre_gs_inst)
group_report(results_df)
```

	param_type	param_set	mean_score	mean_std
0	param_n_estimators	50	0.390483	0.336001
1	param_n_estimators	100	0.581205	0.341753
2	param_min_samples_leaf	2	0.357409	0.357866
3	param_min_samples_leaf	3	0.585257	0.290128
4	param_min_samples_leaf	5	0.769313	0.059378
5	param_min_samples_leaf	10	0.374222	0.396351
6	param_max_features	1	0.454349	0.367366
7	param_max_features	log2	0.537242	0.335230
8	param_max_depth	3	0.628242	0.274399
9	param_max_depth	5	0.469320	0.347066
10	param_max_depth	7	0.318936	0.437246
11	param_max_depth	10	0.459121	0.393417
12	param_loss	huber	0.457164	0.381814
13	param_loss	ls	0.534779	0.321590
14	param_learning_rate	0.0001	-0.021964	0.028927
15	param_learning_rate	0.001	0.059344	0.048931
16	param_learning_rate	0.01	0.456646	0.060294
17	param_learning_rate	0.05	0.757644	0.077207
18	param_learning_rate	0.1	0.771754	0.032609
19	param_learning_rate	0.3	0.811794	0.009037

شکل (۹-۱۸): مقادیر تخمین ها

	param_type	param_set	mean_score	mean_std
0	param_n_estimators	50	0.766376	0.027572
1	param_n_estimators	100	0.793485	0.019273
2	param_min_samples_leaf	3	0.787507	0.024836
3	param_min_samples_leaf	4	0.782410	0.026376
4	param_max_features	0.5	0.790456	0.021662
5	param_max_features	1	0.794523	0.018081
6	param_max_features	sqrt	0.766816	0.029156
7	param_max_depth	2	0.762965	0.021327
8	param_max_depth	3	0.795316	0.014759
9	param_max_depth	4	0.804770	0.010943
10	param_loss	huber	0.783377	0.025044
11	param_loss	ls	0.786593	0.027514
12	param_learning_rate	0.2	0.798660	0.018812
13	param_learning_rate	0.25	0.775424	0.029915
14	param_learning_rate	0.3	0.796568	0.011777
15	param_learning_rate	0.4	0.790480	0.017389

شکل (۹-۱۹): مقادیر تخمین‌های دیگر

با استفاده از این اطلاعات، جستجوی تصادفی دیگری را امتحان کنید

```
param_dist = {'max_features' : [0.4, 0.5, 0.6],
'max_depth' : [5,6],
'min_samples_leaf' : [4,5],
'n_estimators': [300],
'learning_rate' : [0.3],
'loss' : ['ls','huber']
}
```

تیجه را با rs_gbt، برای آخرین بار با ۴۰۰۰ تخمین زننده، آموزش را انجام می‌دهیم

```
rs_gbt = GradientBoostingRegressor(warm_start=True,
max_features = 0.5,
min_samples_leaf = 4,
learning_rate=0.3,
max_depth = 6,
n_estimators = 4000,loss = 'huber')
rs_gbt.fit(X_train, y_train)
```

```
y_pred = rs_gbt.predict(X_test)
from sklearn.metrics import r2_score, mean_absolute_error
print ("R-squared",r2_score(y_test, y_pred))
print ("MAE : ",mean_absolute_error(y_test, y_pred))
print ("MAPE : ",(np.abs(y_test - y_pred)/y_test).mean())
output:
R-squared 0.8379361858488144
MAE : 0.3098475143393069
MAPE : 0.17763868162056828
```

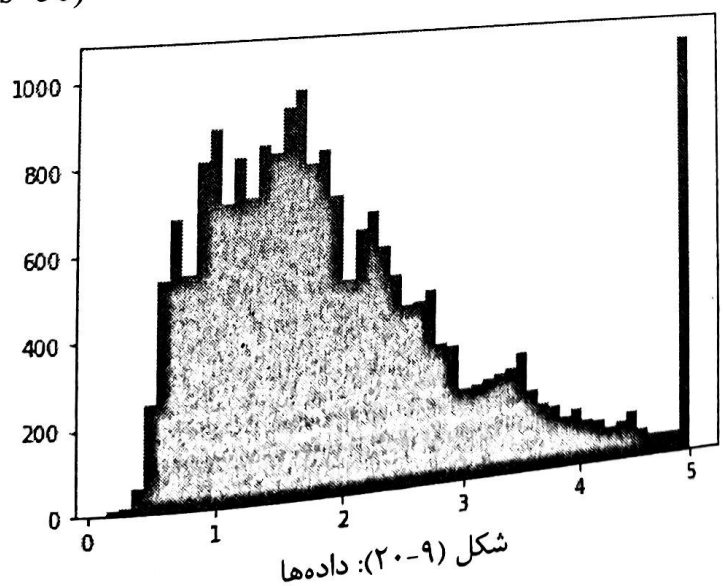
یافتن بهترین پارامترهای کلاسه‌بند گرادیان صعودی

طبقه‌بندی با استفاده از gradient boosting tree بسیار مشابه رگرسیونی است که ما انجام داده‌ایم. دوباره، موارد زیر را انجام خواهیم داد: بهترین پارامترهای کلاسه بند گرادیان صعودی را پیدا کنید. اینها همان رگرسیون تقویت کننده گرادیان هستند، با این تفاوت که گزینه‌های تابع ضرر متفاوت هستند. پارامترها به شرح زیر است:

- max_features
- max_depth
- min_samples_leaf
- learning_rate
- loss

یک تخمین زننده با بهترین پارامتر اما تعداد بیشتری درخت اجرا می‌کنیم. در کد زیر، به تغییر تابع ضرر به نام deviance توجه داشته باشید. برای انجام کلاسه‌بندی، از یک متغیر باینری استفاده خواهیم کرد. نمودار زیر

```
plt.hist(y, bins=50)
```



شکل (۹-۲۰): داده‌ها

در سمت راست، به نظر می‌رسد ناهنجاری وجود دارد: مقدار زیادی داده در ۵ وجود دارد. شاید ما بخواهیم آن مجموعه را جدا کنیم و جداگانه تحلیل کنیم. به عنوان بخشی از این فرایند، ممکن است بخواهیم تعیین کنیم که آیا یک نقطه باید به مجموعه ناهنجاری تعلق داشته باشد یا خیر. یک کلاسه بند برای جدا کردن نقاطی که y برابر با پنج است یا بزرگ‌تر است، ایجاد خواهیم کرد.

ابتدا مجموعه را به آموزش و آزمایش تقسیم کنید. یک متغیر باینری ایجاد کنید، اگر متغیر هدف ۵ یا بیشتر باشد برابر ۱ و در غیر این صورت برابر صفر باشد. توجه داشته باشید که اگر متغیر باینری ۱ باشد، آن مجموعه مربوط به مجموعه غیر عادی است. سپس، از ابعاد X_train برای تقسیم متغیر باینری به y_train_binned و y_test_binned استفاده کنید.

```
bins = np.arange(6)
binned_y = np.digitize(y, bins)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2,stratify=binned_y)
y_binary = np.where(y >= 5, 1,0)
train_shape = X_train.shape[0]
y_train_binned = y_binary[:train_shape]
y_test_binned = y_binary[train_shape:]
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import RandomizedSearchCV
param_dist = {'max_features' : ['log2',0.5,1.0],
'max_depth' : [2,3,6],
'min_samples_leaf' : [1,2,3,10],
'n_estimators': [100],
'learning_rate' : [0.1,0.2,0.3,1],
'loss' : ['deviance']
}
pre_gs_inst
=RandomizedSearchCV(GradientBoostingClassifier(warm_start=True),
param_distributions = param_dist,cv=3,n_iter = 10, n_jobs=1)
pre_gs_inst.fit(X_train, y_train_binned)
pre_gs_inst.best_params_
output: {'n_estimators': 100,
'min_samples_leaf': 10,
'max_features': 0.5,
'max_depth': 2,
'loss': 'deviance',
'learning_rate': 0.3}
```

تعداد تخمین زننده‌ها را افزایش دهید و تخمین زننده نهایی را آموزش دهید:

```
gbc = GradientBoostingClassifier(**{'learning_rate': 0.2,
'loss': 'deviance',
'max_depth': 2,
'max_features': 1.0,
'min_samples_leaf': 2,
'n_estimators': 1000, 'warm_start':True}).fit(X_train,
y_train_binned)
y_pred = gbc.predict(X_test)
from sklearn.metrics import accuracy_score
accuracy_score(y_test_binned, y_pred)
output: 0.935562015503876
```

الگوریتم، کلاسه بند باینری، در تعیین اینکه آیا خانه به مجموعه ناهنجار تعلق دارد، حدود ۹۴٪ دقیق است. بهینه‌سازی ابرپارامتر کلاسه‌بند گرادیان صعودی با پارامترهای مهم همانند رگرسیون افزایش گرادیان است.

تنظیم رگرسیون AdaBoost

پارامترهای مهمی که در یک رگرسیونگر AdaBoost تغییر می‌کند، learning_rateand loss است. همانند الگوریتم‌های قبلی، ما یک جستجوی پارامتر تصادفی انجام می‌دهیم تا بهترین امتیاز الگوریتم را پیدا کنیم.

الگوریتم و جستجوی تصادفی شبکه را انتخاب و توزیع پارامتر تصادفی را امتحان می‌کنیم:

```
from sklearn.ensemble import AdaBoostRegressor
from sklearn.model_selection import RandomizedSearchCV
param_dist = {
'n_estimators': [50, 100],
'learning_rate' : [0.01,0.05,0.1,0.3,1],
'loss' : ['linear', 'square', 'exponential']
}
pre_gs_inst = RandomizedSearchCV(AdaBoostRegressor(),
param_distributions = param_dist,
cv=3,
n_iter = 10,
n_jobs= 1)
pre_gs_inst.fit(X_train, y_train)
```

بهترین پارامتر

```
pre_gs_inst.best_params_
output: {'n_estimators': 100, 'loss': 'exponential', 'learning_rate': 0.05}
```

حال جستجوی تصادفی دیگری را با توزیع پارامتر متفاوت انجام می‌دهیم:

```
param_dist = {
'n_estimators': [100],
'learning_rate' : [0.04,0.045,0.05,0.055,0.06],
'loss' : ['linear']
}
```

فرهنگ لغتی را که بهترین پارامترها را در اختیار دارد کپی کرده، و تعداد تخمین زننده‌ها را به ۳۰۰۰ مورد می‌رسانیم:

```
import copy
ada_best = copy.deepcopy(pre_gs_inst.best_params_)
ada_best['n_estimators'] = 3000
```

مدل نهایی Adaboost را آموزش دهید

```
rs_ada = AdaBoostRegressor(**ada_best)
rs_ada.fit(X_train, y_train)
```

عملکرد مدل را روی مجموعه آزمایش اندازه‌گیری می‌کنیم:

```
y_pred = rs_ada.predict(X_test)
from sklearn.metrics import r2_score, mean_absolute_error
print ("R-squared",r2_score(y_test, y_pred))
print ("MAE : ",mean_absolute_error(y_test, y_pred))
print ("MAPE : ",(np.abs(y_test - y_pred)/y_test).mean())
output:
R-squared -0.022635868577780105
MAE : 1.0405546612411347
MAPE : 0.8384715562542282
```

ما بهترین پارامترها را برای چند الگوریتم پیدا کرده‌ایم. در اینجا جدول خلاصه‌ای از پارامترها برای بهینه‌سازی برای هر الگوریتم تحت اعتبارسنجی وجود دارد. پیشنهاد می‌شود بهینه‌سازی این پارامترها را شروع کنیم.

جدول (۹-۱): جدول پارامترها

Important Variables to Optimize for Various Ensembles/Models	
Decision tree	max_depth
Random forest	max_features, min_samples_leaf, oob_score
Extra trees	max_features, min_samples_leaf, oob_score (same as Random forest)
Bagging ensembles	max_features, max_samples, oob_score, any parameters of the base estimators
Gradient boosting trees	max_features, max_depth, min_samples_leaf, learning_rate, loss
Ada boost	learning_rate, loss, any parameters of the base estimators

نوشتن یک جمع کننده انباشته (stacking aggregator)

در این بخش، ما یک جمع کننده پشته را با scikit-learn می‌نویسیم. یک جمع کننده پشته مدل‌هایی از انواع کاملاً متفاوت را با هم مخلوط می‌کند. در بسیاری از الگوریتم‌های Ensemble، نمونه‌های ترکیبی از همان نوع (معمولا درختان تصمیم) را دیده‌ایم. فرایند اساسی در جمع کننده پشته‌ای این است که ما از پیش‌بینی چندین الگوریتم یادگیری ماشین به عنوان ورودی برای آموزش الگوریتم یادگیری ماشین دیگر استفاده می‌کنیم.

با جزئیات بیشتر، ما دو یا چند الگوریتم یادگیری ماشین را با استفاده از یک جفت مجموعه X و y آموزش می‌دهیم (X_1، y_1). سپس ما بر روی مجموعه X دوم X_stack، y_pred_1، y_pred_2 و غیره پیش‌بینی می‌کنیم. این پیش‌بینی‌ها، y_pred_1 و y_pred_2، با خروجی آموزش y_stack، به ورودی الگوریتم یادگیری ماشین تبدیل می‌شوند. سرانجام، می‌توان خطا را روی یک مجموعه ورودی سوم، X_3 و یک مجموعه، y_3 اندازه‌گیری کرد.

بار دیگر داده‌های مجموعه داده مسکن کالیفرنیا را بارگیری می‌کنیم.

```
from __future__ import division
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
cali_housing = fetch_california_housing()
X = cali_housing.data
y = cali_housing.target
bins = np.arange(6)

from sklearn.model_selection import train_test_split
binned_y = np.digitize(y, bins)
from sklearn.ensemble import RandomForestRegressor,AdaBoostRegressor,
ExtraTreesRegressor, GradientBoostingRegressor
from sklearn.model_selection import GridSearchCV
```

اکنون با دو بار استفاده از train_test_split جفت، X و y را به سه X،y ، جفت ورودی و خروجی تقسیم می‌کنیم.

```
X_train_prin, X_test_prin, y_train_prin, y_test_prin =train_test_split(X,
y,test_size=0.2,stratify=binned_y)
binned_y_train_prin = np.digitize(y_train_prin, bins)
X_1, X_stack, y_1, y_stack = train_test_split(X_train_prin,
```

```
y_train_prin,
test_size=0.33,
stratify=binned_y_train_prin )
```

با استفاده از RandomizedSearchCV، بهترین پارامترها را برای اولین الگوریتم‌های جمع کننده پشته پیدا می‌کنیم، در این مورد یک الگوریتم bagging متشکل از چندین مدل نزدیک‌ترین همسایه knn می‌باشد.

```
from sklearn.ensemble import BaggingRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import RandomizedSearchCV
param_dist = {
'max_samples': [0.5,1.0],
'max_features' : [0.5,1.0],
'oob_score' : [True, False],
'base_estimator__n_neighbors': [3,5],
'n_estimators': [100]
}
```

با استفاده از بهترین پارامترها، baggin regressor را با استفاده از تخمین زننده‌های زیاد (۳۰۰۰ مورد) آموزش می‌دهیم

```
rs_bag = BaggingRegressor(**{'max_features': 0.5,
'max_samples': 0.5,
'n_estimators': 3000,
'oob_score': False,
'base_estimator': KNeighborsRegressor(n_neighbors=3)})
rs_bag.fit(X_1, y_1)
```

همین روند را برای الگوریتم گرادیان صعودی در X_1، y_1 انجام دهید

```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import RandomizedSearchCV
param_dist = {'max_features' : ['log2',0.4,0.5,0.6,1.0],
'max_depth' : [2,3, 4, 5,6, 7, 10],
'min_samples_leaf' : [1,2, 3, 4, 5, 10],
'n_estimators': [50, 100],
'learning_rate' : [0.01,0.05,0.1,0.25,0.275,0.3,0.325],
'loss' : ['ls','huber']
}
pre_gs_inst
=RandomizedSearchCV(GradientBoostingRegressor(warm_start=True),
param_distributions = param_dist,cv=3,n_iter = 30, n_jobs=1)
pre_gs_inst.fit(X_1, y_1)
```

بهترین مجموعه پارامترها را با تخمین زن‌های بیشتری آموزش می‌دهیم

```
gbt_inst = GradientBoostingRegressor(**{'learning_rate': 0.05,
'loss': 'huber',
'max_depth': 10,
'max_features': 0.4,
'min_samples_leaf': 5,
'n_estimators': 3000,
'warm_start': True}).fit(X_1, y_1)
```

پیش‌بینی را انجام می‌دهیم

```
y_pred_bag = rs_bag.predict(X_stack)
y_pred_gbt = gbt_inst.predict(X_stack)
```

معیارهای bagging regressor را مشاهده کنید

```
from sklearn.metrics import r2_score, mean_absolute_error
print ("R-squared",r2_score(y_stack, y_pred_bag))
print ("MAE : ",mean_absolute_error(y_stack, y_pred_bag))
print ("MAPE : ",(np.abs(y_stack- y_pred_bag)/y_stack).mean())
output:
R-squared 0.53541114283572 27
MAE : 0.6042100936343067
MAPE : 0.39718844745619397
```

معیارهای گرادیان را مشاهده کنید

```
from sklearn.metrics import r2_score, mean_absolute_error
print ("R-squared",r2_score(y_stack, y_pred_gbt))
print ("MAE : ",mean_absolute_error(y_stack, y_pred_gbt))
print ("MAPE : ",(np.abs(y_stack - y_pred_gbt)/y_stack).mean())
output:
R-squared 0.8432651102752469
MAE : 0.2940685268905751
MAPE : 0.1629332778468931
```

حال آن‌ها را در یک جدول مقایسه می‌کنیم

```
y_pred_bag = rs_bag.predict(X_stack)
y_pred_gbt = gbt_inst.predict(X_stack)
preds_df = pd.DataFrame(columns = ['bag', 'gbt'])
preds_df['bag'] = y_pred_bag
preds_df['gbt'] = y_pred_gbt
preds_df
```

	bag	gbt
0	1.180689	0.700618
1	1.783200	1.492180
2	2.089861	2.111812
3	1.774681	1.897128
4	2.670214	2.810850
...	...	...
5444	1.357155	1.001409
5445	1.923348	2.086613
5446	1.570228	0.645072
5447	2.396953	2.769171
5448	1.820433	1.530299

5449 rows × 2 columns

شکل (۹-۲۱): امتیاز گرادیان صعودی و bagging

به همبستگی ستون‌های پیش‌بینی نگاه کنید. ستون‌ها در ارتباط هستند، اما کاملاً مناسب نیستند. وضعیت ایده آل این است که الگوریتم‌ها کاملاً با هم همبستگی نداشته باشند و هر دو عملکرد خوبی داشته باشند. در این حالت، bagging regressorای تقریباً به اندازه gradient boosting خوب عمل نمی‌کند

```
preds_df.corr()
```

	bag	gbt
bag	1.000000	0.910385
gbt	0.910385	1.000000

شکل (۹-۲۲): نمایش همبستگی گرادیان صعودی و bagging

اکنون یک جستجوی تصادفی با الگوریتم سوم انجام دهید. این الگوریتم پیش‌بینی‌های دو مورد اول را به عنوان ورودی در نظر می‌گیرد. برای پیش‌بینی‌های دو الگوریتم دیگر، از یک رگرسیون درخت استفاده خواهیم کرد

```
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.model_selection import RandomizedSearchCV
param_dist = {'max_features' : ['sqrt','log2',1.0],
'min_samples_leaf' : [1, 2, 3, 7, 11],
'n_estimators': [50, 100],'oob_score': [True, False]}

pre_gs_inst
=RandomizedSearchCV(ExtraTreesRegressor(warm_start=True,bootstrap=True),
param_distributions = param_dist,
cv=3,
n_iter = 15)
pre_gs_inst.fit(preds_df.values, y_stack)
import copy
param_dict = copy.deepcopy(pre_gs_inst.best_params_)
param_dict['n_estimators'] = 2000
param_dict['warm_start'] = True
param_dict['bootstrap'] = True
param_dict['n_jobs'] = -1
param_dict
{'bootstrap': True,
'max_features': 1.0,
'min_samples_leaf': 11,
'n_estimators': 2000,
'n_jobs': -1,
'oob_score': False,
'warm_start': True}
```

از y_stack به عنوان داده‌های هدف استفاده می‌کنیم

```
final_etr = ExtraTreesRegressor(**param_dict)
final_etr.fit(preds_df.values, y_stack)
```

برای بررسی عملکرد کلی جمع کننده پشته، شما به تابعی نیاز دارید که یک مجموعه X را به عنوان ورودی در نظر بگیرد و با استفاده از رگرسور bagging و گرادیان صعودی پیش‌بینی را انجام دهد.

```
def handle_X_set(X_train_set):
 y_pred_bag = rs_bag.predict(X_train_set)
 y_pred_gbt = gbt_inst.predict(X_train_set)
 preds_df = pd.DataFrame(columns = ['bag', 'gbt'])
 preds_df['bag'] = y_pred_bag
 preds_df['gbt'] = y_pred_gbt
 return preds_df.values
def predict_from_X_set(X_train_set):
```

```
    return final_etr.predict(handle_X_set(X_train_set))
y_pred = predict_from_X_set(X_test_prin)
```

حال کارایی مدل را اندازه می‌گیریم

```
from sklearn.metrics import r2_score, mean_absolute_error
print ("R-squared",r2_score(y_test_prin, y_pred))
print ("MAE : ",mean_absolute_error(y_test_prin, y_pred))
print ("MAPE : ",(np.abs(y_test_prin- y_pred)/y_test_prin).mean())
output:
R-squared 0.8364218435101 78
MAE : 0.3017126102777359
MAPE : 0.16201787045758292
```

معیار R-squared اندکی بهبود یافت و ما برای آن پیشرفت جزئی، تلاش بسیار کردیم. برای قرار دادن بسیاری از تخمین زن‌ها که در درون پشته همبسته نیستند، از روش‌های بهتر استفاده می‌کنیم. علاوه بر این، می‌توانیم مهندسی ویژگی‌ها را انجام دهیم. همچنین می‌توانید الگوریتم‌های مختلفی را برای ورودی‌های مختلف امتحان کنید. دو ستون عرض و طول برای جنگل‌های تصادفی مناسب است و ورودی‌های دیگر می‌توانند با الگوریتم خطی به خوبی مدل شوند.

ما می‌توانیم الگوریتم‌های مختلفی را روی مجموعه داده جستجو کنیم. ما برای این مجموعه داده روی الگوریتم‌های پیچیده و با واریانس بالا تمرکز کردیم. می‌توان الگوریتم‌های ساده با بایاس زیاد را امتحان کرد. این الگوریتم‌های جایگزین می‌توانند به جمع کننده پشته‌ای که در انتها استفاده کردیم، کمک کنند.

در رابطه با استکر، می‌توانید مجموعه X_stacker را با crossvalidation تکرار کنید تا از مجموعه آموزش بیشترین استفاده را ببرید.